

Playwright Trace CLI

Inspect .zip trace files produced by Playwright tests without opening a browser.

Workflow

1. Start with `trace open <trace.zip>` to extract the trace and see its metadata.
2. Use `trace actions` to see all actions with their action IDs.
3. Use `trace action <action-id>` to drill into a specific action — see parameters, logs, source location, and available snapshots.
4. Use `trace requests`, `trace console`, or `trace errors` for cross-cutting views.
5. Use `trace snapshot <action-id>` to get the DOM snapshot, or run a browser command against it.
6. Use `trace close` to remove the extracted trace data when done.

All commands after `open` operate on the currently opened trace — no need to pass the trace file again. Opening a new trace replaces the previous one.

Commands

Open a trace

Extract trace and show metadata: browser, viewport, duration, action/error counts
`npx playwright trace open <trace.zip>`

Close a trace

Remove extracted trace data
`npx playwright trace close`

Actions

List all actions as a tree with action IDs and timing
`npx playwright trace actions`

Filter by action title (regex, case-insensitive)
`npx playwright trace actions --grep "click"`

Only failed actions
`npx playwright trace actions --errors-only`

Action details

Show full details for one action: params, result, logs, source, snapshots
npx playwright trace action <action-id>

The action command displays available snapshot phases (before, input, after) and the exact command to extract them.

Requests

All network requests: method, status, URL, duration, size
npx playwright trace requests

Filter by URL pattern
npx playwright trace requests --grep "api"

Filter by HTTP method
npx playwright trace requests --method POST

Only failed requests (status >= 400)
npx playwright trace requests --failed

Request details

Show full details for one request: headers, body, security
npx playwright trace request <request-id>

Console

All console messages and stdout/stderr
npx playwright trace console

Only errors
npx playwright trace console --errors-only

Only browser console (no stdout/stderr)
npx playwright trace console --browser

Only stdout/stderr (no browser console)
npx playwright trace console --stdio

Errors

All errors with stack traces and associated actions
npx playwright trace errors

Snapshots

The snapshot command loads the DOM snapshot for an action into a headless browser and runs a single browser command against it. Without a browser command, it returns the accessibility snapshot.

Get the accessibility snapshot (default)

```
npx playwright trace snapshot <action-id>
```

Use a specific phase

```
npx playwright trace snapshot <action-id> --name before
```

Run eval to query the DOM

```
npx playwright trace snapshot <action-id> -- eval "document.title"
```

```
npx playwright trace snapshot <action-id> -- eval "document.querySelector('#error'"
```

Eval on a specific element ref (from the snapshot)

```
npx playwright trace snapshot <action-id> -- eval "el => el.getAttribute('data-t'"
```

Take a screenshot of the snapshot

```
npx playwright trace snapshot <action-id> -- screenshot
```

Redirect output to a file

```
npx playwright trace snapshot <action-id> -- eval "document.body.outerHTML" --fi
```

```
npx playwright trace snapshot <action-id> -- screenshot --filename=screenshot.png
```

Only three browser commands are useful on a frozen snapshot: snapshot, eval, and screenshot.

Attachments

List all trace attachments

```
npx playwright trace attachments
```

Extract an attachment by its number

```
npx playwright trace attachment 1
```

```
npx playwright trace attachment 1 -o out.png
```

Typical investigation

1. Open the trace and see what's inside

```
npx playwright trace open test-results/my-test/trace.zip
```

2. What actions ran?

```
npx playwright trace actions
```

3. Which action failed?

```
npx playwright trace actions --errors-only
```

4. What went wrong?

```
npx playwright trace action 12
```

5. What did the page look like at that moment?

```
npx playwright trace snapshot 12
```

6. Query the DOM for more detail

```
npx playwright trace snapshot 12 -- eval "document.querySelector('.error-message
```

7. Any relevant network failures?

```
npx playwright trace requests --failed
```

8. Any console errors?

```
npx playwright trace console --errors-only
```